# Análisis Exploratorio de Datos (EDA)

## UI-PRMD - Physical Rehabilitation Movement Dataset

**Dataset:** University of Idaho - Physical Rehabilitation Movement Dataset

Este notebook realiza un análisis exploratorio completo del dataset UI-PRMD, que contiene datos de movimientos de rehabilitación física capturados mediante sistemas Kinect y Vicon.

## Descripción del Dataset

Leemos la información del metadato para entender el contexto del dataset.

Nombre: UI-PRMD (University of Idaho – Physical Rehabilitation Movement Dataset)
Publicado en 2018 por la Universidad de Idaho.
Puede encontrarse en: https://opendatalab.com/OpenDataLab/UI-PRMD
El UI-PRMD (University of Idaho – Physical Rehabilitation Movement Dataset) es un conjunto de datos diseñado para el análisis de movimientos humanos orientados a ejercicios de rehabilitación física. Este dataset fue desarrollado por investigadores de la University of Idaho con el objetivo de proporcionar datos de referencia para el estudio de técnicas de análisis biomecánico, reconocimiento de actividades humanas y evaluación automática de ejercicios de rehabilitación.

### Importar Librerías

In [46]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from pathlib import Path
import yaml
import warnings

warnings.filterwarnings('ignore')

# Configuración de visualización
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)

# Tamaño de figuras por defecto
plt.rcParams['figure.figsize'] = (14, 6)
```

```
plt.rcParams['font.size'] = 10

# Colores personalizados
PRIMARY_COLOR = '#0098cd'
SECONDARY_COLOR = '#FFC107'
```

# Exploración de la Estructura de Archivos

Exploramos la organización de directorios y archivos del dataset.

In [47]:
```
# Estructura del dataset
base_path = Path('./dataset')

# Categorías principales
categories = [
    'Movements',
    'Incorrect Movements',
    'Segmented Movements',
    'Incorrect Segmented Movements'
]

# Sistemas de captura
systems = ['Kinect', 'Vicon']

# Tipos de datos
data_types = ['Angles', 'Positions']

print("ESTRUCTURA DEL DATASET")
print("="*60)

# Recopilar estadísticas
file_stats = []

for category in categories:
    category_path = base_path / category / category
    if category_path.exists():
        print(f"\n{category}")
        for system in systems:
            for data_type in data_types:
                data_path = category_path / system / data_type
                if data_path.exists():
                    files = list(data_path.glob('*.txt'))
                    file_stats.append({
                        'Categoría': category,
                        'Sistema': system,
                        'Tipo': data_type,
                        'Cantidad': len(files)
                    })
                    print(f"   └─ {system}/{data_type}: {len(files)} archivos")

# Crear DataFrame de estadísticas
df_stats = pd.DataFrame(file_stats)
print("\n\nRESUMEN DE ARCHIVOS")
```

```
print("="*60)
print(df_stats.pivot_table(index=['Categoría'], columns=['Sistema', 'Tipo'], values
```

```
ESTRUCTURA DEL DATASET
============================================================

Movements
   └─ Kinect/Angles: 100 archivos
   └─ Kinect/Positions: 100 archivos
   └─ Vicon/Angles: 99 archivos
   └─ Vicon/Positions: 99 archivos

Incorrect Movements
   └─ Kinect/Angles: 100 archivos
   └─ Kinect/Positions: 100 archivos
   └─ Vicon/Angles: 99 archivos
   └─ Vicon/Positions: 99 archivos

Segmented Movements
   └─ Kinect/Angles: 1000 archivos
   └─ Kinect/Positions: 1000 archivos
   └─ Vicon/Angles: 990 archivos
   └─ Vicon/Positions: 990 archivos

Incorrect Segmented Movements
   └─ Kinect/Angles: 1000 archivos
   └─ Kinect/Positions: 1000 archivos
   └─ Vicon/Angles: 990 archivos
   └─ Vicon/Positions: 990 archivos


RESUMEN DE ARCHIVOS
============================================================
Sistema                        Kinect            Vicon
Tipo                           Angles Positions Angles Positions
Categoría
Incorrect Movements             100.0     100.0   99.0      99.0
Incorrect Segmented Movements  1000.0    1000.0  990.0     990.0
Movements                       100.0     100.0   99.0      99.0
Segmented Movements            1000.0    1000.0  990.0     990.0
```

# Funciones de Carga de Datos

Definimos funciones auxiliares para cargar y procesar los datos.

In [48]:
```
def load_data_file(file_path):
    """
    Carga un archivo de datos del dataset.

    Args:
        file_path: Ruta al archivo

    Returns:
        DataFrame con los datos cargados
    """
```

```python
    try:
        # Leer el archivo (valores separados por espacios múltiples)
        data = pd.read_csv(file_path, delim_whitespace=True, header=None)
        return data
    except Exception as e:
        print(f"Error cargando {file_path}: {e}")
        return None

def get_file_info(filename):
    """
    Extrae información del nombre del archivo.

    Args:
        filename: Nombre del archivo (ej: m01_s01_angles.txt)

    Returns:
        Dict con información del movimiento, sujeto, tipo
    """
    parts = filename.replace('.txt', '').split('_')
    info = {
        'filename': filename,
        'movement': parts[0] if len(parts) > 0 else None,
        'subject': parts[1] if len(parts) > 1 else None,
        'type': parts[2] if len(parts) > 2 else None,
        'incorrect': 'inc' in filename
    }
    return info

def load_category_data(category, system='Kinect', data_type='Angles', limit=None):
    """
    Carga todos los archivos de una categoría específica.

    Args:
        category: Nombre de la categoría (ej: 'Movements')
        system: 'Kinect' o 'Vicon'
        data_type: 'Angles' o 'Positions'
        limit: Límite de archivos a cargar (None = todos)

    Returns:
        Lista de tuplas (info, dataframe)
    """
    data_path = "dataset" / Path(category) / category / system / data_type
    if not data_path.exists():
        print(f"Ruta no encontrada: {data_path}")
        return []

    files = list(data_path.glob('*.txt'))
    if limit:
        files = files[:limit]

    loaded_data = []
    for file_path in files:
        info = get_file_info(file_path.name)
        data = load_data_file(file_path)
        if data is not None:
            loaded_data.append((info, data))
```

```
    print(f"Cargados {len(loaded_data)} archivos de {category}/{system}/{data_type}
    return loaded_data
```

# Carga de Datos de Muestra

Cargamos ejemplos de movimientos correctos e incorrectos para análisis inicial.

In [49]:
```
# Cargar datos de movimientos correctos (5 primeros archivos)
print("Cargando movimientos correctos...")
correct_data = load_category_data('Movements', system='Kinect', data_type='Angles',

# Cargar datos de movimientos incorrectos (5 primeros archivos)
print("\nCargando movimientos incorrectos...")
incorrect_data = load_category_data('Incorrect Movements', system='Kinect', data_ty

# Analizar un archivo de muestra
if correct_data:
    print("\n\nANÁLISIS DE ARCHIVO DE MUESTRA (Movimiento Correcto)")
    print("="*80)
    sample_info, sample_data = correct_data[0]
    print(f"Archivo: {sample_info}")
    print(f"\nForma del DataFrame: {sample_data.shape}")
    print(f"- Número de frames (filas): {sample_data.shape[0]}")
    print(f"- Número de features (columnas): {sample_data.shape[1]}")
    print(f"\nPrimeras 5 filas:")
    display(sample_data.head())

    print(f"\nEstadísticas descriptivas:")
    display(sample_data.describe())

if incorrect_data:
    print("\n\nANÁLISIS DE ARCHIVO DE MUESTRA (Movimiento Incorrecto)")
    print("="*80)
    sample_info_inc, sample_data_inc = incorrect_data[0]
    print(f"Archivo: {sample_info_inc}")
    print(f"\nForma del DataFrame: {sample_data_inc.shape}")
    print(f"- Número de frames (filas): {sample_data_inc.shape[0]}")
    print(f"- Número de features (columnas): {sample_data_inc.shape[1]}")
    print(f"\nPrimeras 5 filas:")
    display(sample_data_inc.head())
```

```
Cargando movimientos correctos...
Cargados 5 archivos de Movements/Kinect/Angles

Cargando movimientos incorrectos...
Cargados 5 archivos de Incorrect Movements/Kinect/Angles


ANÁLISIS DE ARCHIVO DE MUESTRA (Movimiento Correcto)
================================================================================
Archivo: {'filename': 'm01_s01_angles.txt', 'movement': 'm01', 'subject': 's01', 'ty
pe': 'angles', 'incorrect': False}

Forma del DataFrame: (765, 66)
- Número de frames (filas): 765
- Número de features (columnas): 66

Primeras 5 filas:
```

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | -3.20393 | -3.76583 | -2.44245 | 1.75432 | -0.21344 | -0.17643 | 2.42654 | -0.31887 | -0.11283 | 0.00 |
| **1** | -3.22288 | -3.76855 | -2.42007 | 1.76524 | -0.20911 | -0.18931 | 2.43129 | -0.31525 | -0.12114 | 0.00 |
| **2** | -3.26808 | -3.81693 | -2.36485 | 1.79071 | -0.18167 | -0.21305 | 2.44926 | -0.29009 | -0.14569 | 0.00 |
| **3** | -3.29496 | -3.88067 | -2.32399 | 1.79585 | -0.17216 | -0.21459 | 2.45372 | -0.28475 | -0.15052 | 0.00 |
| **4** | -3.31914 | -3.82097 | -2.29318 | 1.79234 | -0.22862 | -0.22321 | 2.43057 | -0.34551 | -0.13442 | 0.00 |

```
Estadísticas descriptivas:
```

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| **count** | 765.000000 | 765.000000 | 765.000000 | 765.000000 | 765.000000 | 765.000000 | 765.000000 |
| **mean** | 19.264091 | -2.816389 | -1.709037 | 2.163053 | -0.939753 | -0.121885 | 2.804717 |
| **std** | 12.795891 | 1.818486 | 1.262833 | 0.976153 | 0.905981 | 0.265983 | 0.787526 |
| **min** | -3.319140 | -8.379260 | -5.965290 | -1.251610 | -3.704410 | -1.517360 | 0.645550 |
| **25%** | 4.866570 | -4.109950 | -2.297840 | 1.770190 | -1.507890 | -0.242850 | 2.386670 |
| **50%** | 24.326700 | -2.888950 | -1.581780 | 2.095100 | -0.867890 | -0.128420 | 2.611600 |
| **75%** | 31.213500 | -1.626470 | -0.980530 | 2.491370 | -0.279710 | -0.004430 | 3.010360 |
| **max** | 35.696750 | 2.293150 | 2.493980 | 8.605370 | 2.223180 | 2.188330 | 6.781000 |

```
ANÁLISIS DE ARCHIVO DE MUESTRA (Movimiento Incorrecto)
================================================================================
Archivo: {'filename': 'm01_s01_angles_inc.txt', 'movement': 'm01', 'subject': 's01',
'type': 'angles', 'incorrect': True}

Forma del DataFrame: (1096, 66)
- Número de frames (filas): 1096
- Número de features (columnas): 66

Primeras 5 filas:
```

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.33375 | -6.35973 | -3.20115 | 1.79158 | -2.48541 | -0.35221 | 2.46296 | -2.65313 | -0.52450 | -0.00 |
| **1** | 0.31835 | -6.27634 | -3.15850 | 1.79180 | -2.31097 | -0.34782 | 2.46333 | -2.46252 | -0.51068 | -0.00 |
| **2** | 0.19561 | -6.15852 | -3.11343 | 1.78329 | -1.90926 | -0.30907 | 2.46235 | -2.03607 | -0.45944 | -0.00 |
| **3** | 0.27488 | -6.17435 | -3.08102 | 1.77435 | -1.71503 | -0.32992 | 2.48036 | -1.82600 | -0.46743 | -0.00 |
| **4** | 0.29562 | -6.02296 | -3.08557 | 1.81365 | -1.69414 | -0.33409 | 2.51372 | -1.79808 | -0.45654 | -0.00 |

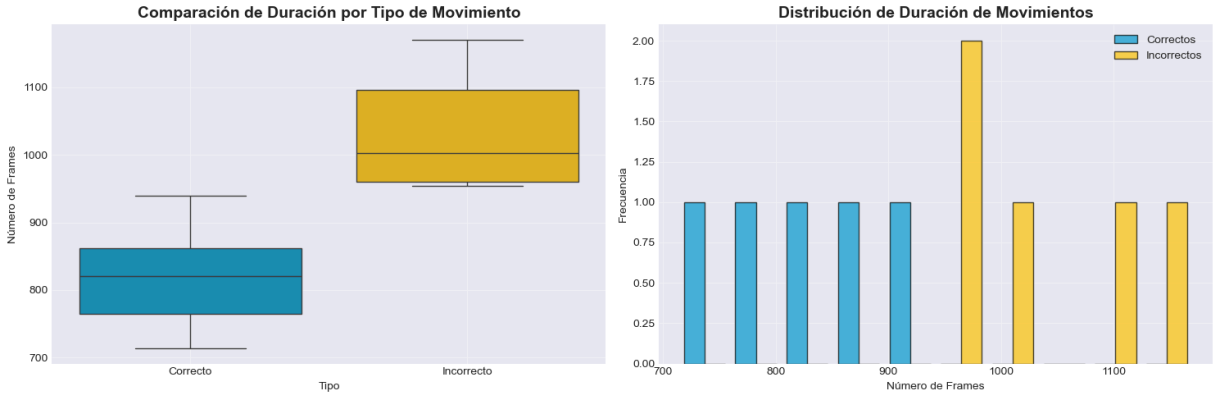
# Comparación de Longitud Temporal

Comparamos la duración (número de frames) entre movimientos correctos e incorrectos.

In [50]:
```
# Estadísticas
print("ESTADÍSTICAS DE DURACIÓN (en frames)")
print("="*60)
print(comparison_df.groupby('Tipo')['Frames'].describe())

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Box plot
sns.boxplot(data=comparison_df, x='Tipo', y='Frames', ax=axes[0], palette=[PRIMARY_
axes[0].set_title('Comparación de Duración por Tipo de Movimiento', fontsize=14, fo
axes[0].set_ylabel('Número de Frames')
axes[0].grid(True, alpha=0.3)

# Histogram
axes[1].hist([correct_lengths, incorrect_lengths], label=['Correctos', 'Incorrectos
             bins=10, alpha=0.7, edgecolor='black', color=[PRIMARY_COLOR, SECONDARY
axes[1].set_xlabel('Número de Frames')
axes[1].set_ylabel('Frecuencia')
axes[1].set_title('Distribución de Duración de Movimientos', fontsize=14, fontweigh
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
# Diferencia promedio
avg_correct = np.mean(correct_lengths)
avg_incorrect = np.mean(incorrect_lengths)
diff_pct = ((avg_incorrect - avg_correct) / avg_correct) * 100

print(f"\nINSIGHT: Los movimientos incorrectos son en promedio {diff_pct:.1f}% {'má
```

```
ESTADÍSTICAS DE DURACIÓN (en frames)
============================================================
            count    mean        std    min    25%    50%    75%    max
Tipo
Correcto      5.0   820.2  86.871745  714.0  765.0  821.0  862.0  939.0
Incorrecto    5.0  1036.6  93.732598  954.0  960.0 1003.0 1096.0 1170.0
```




```
INSIGHT: Los movimientos incorrectos son en promedio 26.4% más largos que los correc
tos
```

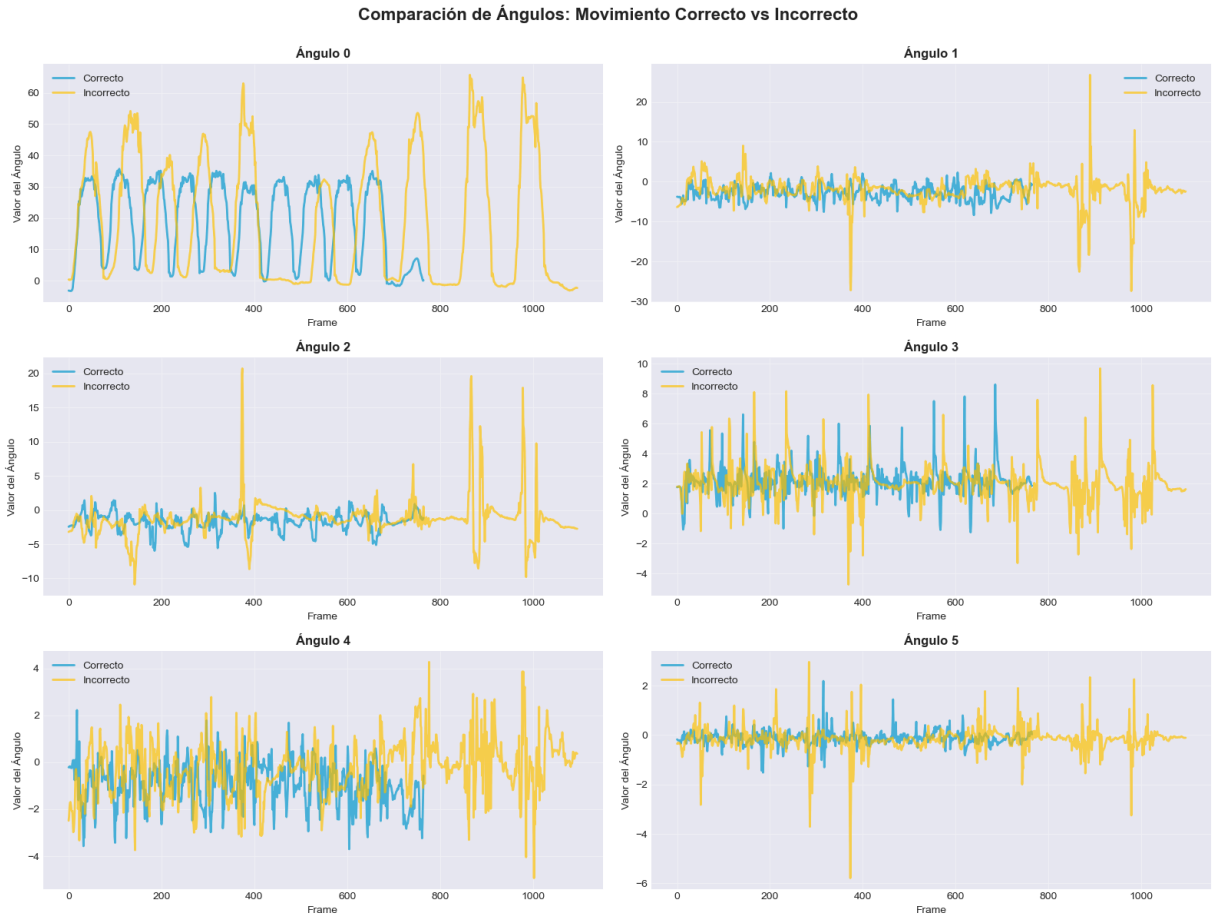
# Visualización de Series Temporales

Visualizamos algunas características (ángulos) a lo largo del tiempo para movimientos correctos e incorrectos.

In [51]:

```
# Crear visualización
fig, axes = plt.subplots(3, 2, figsize=(16, 12))
axes = axes.flatten()

for i, col in enumerate(columns_to_plot):
    if col < correct_sample.shape[1]:
        # Plot correcto
        axes[i].plot(correct_sample[col], label='Correcto', color=PRIMARY_COLOR
        # Plot incorrecto
        if col < incorrect_sample.shape[1]:
            axes[i].plot(incorrect_sample[col], label='Incorrecto', color=SECON

        axes[i].set_title(f'Ángulo {col}', fontsize=12, fontweight='bold')
        axes[i].set_xlabel('Frame')
        axes[i].set_ylabel('Valor del Ángulo')
        axes[i].legend()
        axes[i].grid(True, alpha=0.3)

plt.suptitle('Comparación de Ángulos: Movimiento Correcto vs Incorrecto',
             fontsize=16, fontweight='bold', y=1.00)
plt.tight_layout()
```

```
    plt.show()

print("INSIGHT: Observa las diferencias en los patrones de movimiento entre ejecuci
```




```
INSIGHT: Observa las diferencias en los patrones de movimiento entre ejecuciones cor
rectas e incorrectas
```

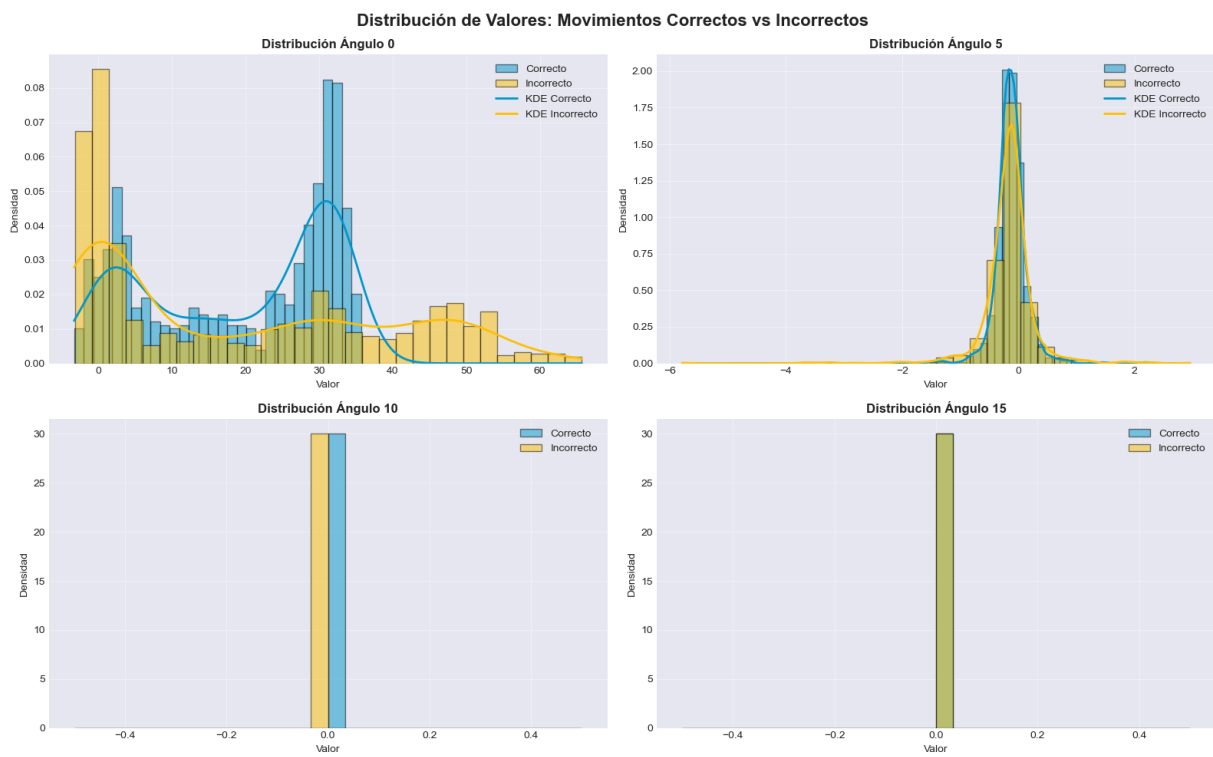
# 8. Análisis de Distribución de Valores

Analizamos la distribución estadística de los valores de los ángulos para identificar patrones y posibles anomalías.

In [52]:

```
    fig, axes = plt.subplots(2, 2, figsize=(16, 10))
    axes = axes.flatten()

    for i, feature in enumerate(features_to_analyze):
        if feature < correct_sample.shape[1]:
            # Histogramas con KDE
            axes[i].hist(correct_sample[feature], bins=30, alpha=0.5, label='Correc
                         color=PRIMARY_COLOR, density=True, edgecolor='black')
            if feature < incorrect_sample.shape[1]:
                axes[i].hist(incorrect_sample[feature], bins=30, alpha=0.5, label='
                             color=SECONDARY_COLOR, density=True, edgecolor='black')

            # KDE plots
            try:
                from scipy import stats
```

```
                kde_correct = stats.gaussian_kde(correct_sample[feature])
                kde_incorrect = stats.gaussian_kde(incorrect_sample[feature])

                x_range = np.linspace(min(correct_sample[feature].min(), incorrect_
                                      max(correct_sample[feature].max(), incorrect_s
                axes[i].plot(x_range, kde_correct(x_range), color=PRIMARY_COLOR, li
                axes[i].plot(x_range, kde_incorrect(x_range), color=SECONDARY_COLOR
            except:
                pass

            axes[i].set_title(f'Distribución Ángulo {feature}', fontsize=12, fontwe
            axes[i].set_xlabel('Valor')
            axes[i].set_ylabel('Densidad')
            axes[i].legend()
            axes[i].grid(True, alpha=0.3)

    plt.suptitle('Distribución de Valores: Movimientos Correctos vs Incorrectos',
                 fontsize=16, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # Estadísticas descriptivas
    print("\nESTADÍSTICAS DESCRIPTIVAS - Ángulo 0:")
    print("\nMovimiento Correcto:")
    print(pd.DataFrame(correct_sample[0]).describe())
    print("\nMovimiento Incorrecto:")
    print(pd.DataFrame(incorrect_sample[0]).describe())
```



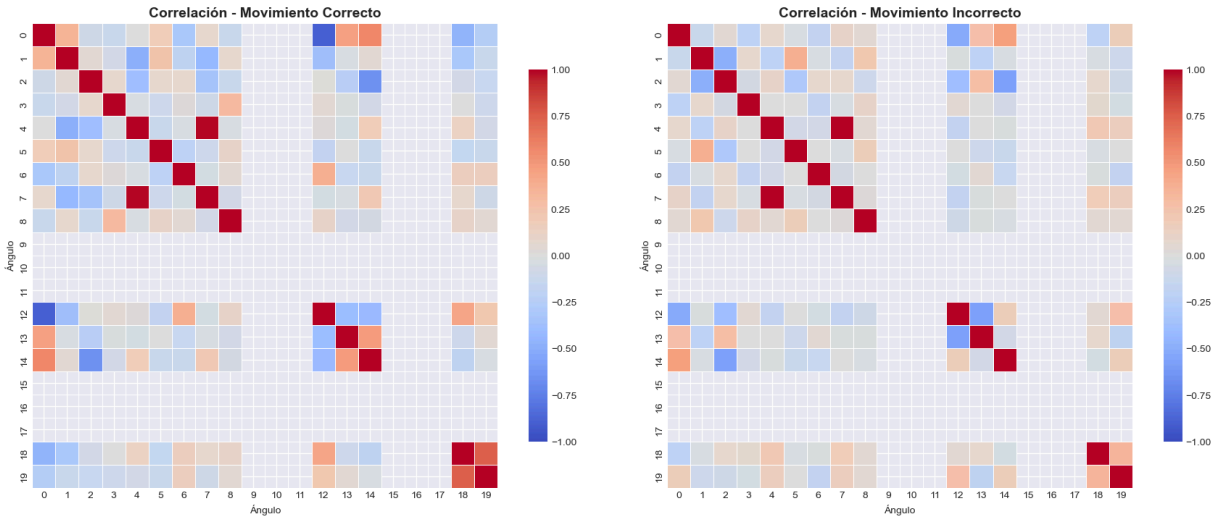
```
ESTADÍSTICAS DESCRIPTIVAS - Ángulo 0:

Movimiento Correcto:
                0
count  765.000000
mean    19.264091
std     12.795891
min     -3.319140
25%      4.866570
50%     24.326700
75%     31.213500
max     35.696750

Movimiento Incorrecto:
                 0
count  1096.000000
mean     18.859755
std      20.211624
min      -3.192260
25%       0.274462
50%      11.562175
75%      36.082882
max      65.724780
```

## Análisis de Correlación entre Ángulos

Analizamos las correlaciones entre diferentes ángulos para entender las relaciones cinemáticas en el movimiento.

In [53]:
```
# Matriz de correlación
if correct_data and incorrect_data:
    _, correct_sample = correct_data[0]
    _, incorrect_sample = incorrect_data[0]

    # Limitar a primeros 20 ángulos para mejor visualización
    n_angles = min(20, correct_sample.shape[1])

    # Calcular correlaciones
    corr_correct = correct_sample.iloc[:, :n_angles].corr()
    corr_incorrect = incorrect_sample.iloc[:, :n_angles].corr()

    # Visualizar
    fig, axes = plt.subplots(1, 2, figsize=(18, 7))

    # Correcto
    sns.heatmap(corr_correct, annot=False, cmap='coolwarm', center=0,
                square=True, linewidths=0.5, cbar_kws={"shrink": 0.8},
                vmin=-1, vmax=1, ax=axes[0])
    axes[0].set_title('Correlación - Movimiento Correcto', fontsize=14, fontweight=
    axes[0].set_xlabel('Ángulo')
    axes[0].set_ylabel('Ángulo')

    # Incorrecto
    sns.heatmap(corr_incorrect, annot=False, cmap='coolwarm', center=0,
```

```
                square=True, linewidths=0.5, cbar_kws={"shrink": 0.8},
                vmin=-1, vmax=1, ax=axes[1])
    axes[1].set_title('Correlación - Movimiento Incorrecto', fontsize=14, fontweigh
    axes[1].set_xlabel('Ángulo')
    axes[1].set_ylabel('Ángulo')

    plt.tight_layout()
    plt.show()

    # Diferencia de correlaciones
    corr_diff = np.abs(corr_correct - corr_incorrect)

    print("\nANÁLISIS DE DIFERENCIAS EN CORRELACIÓN:")
    print(f"Diferencia máxima en correlación: {corr_diff.max().max():.3f}")
    print(f"Diferencia promedio: {corr_diff.mean().mean():.3f}")

    # Encontrar las correlaciones más diferentes
    corr_diff_flat = corr_diff.values[np.triu_indices_from(corr_diff, k=1)]
    if len(corr_diff_flat) > 0:
        print(f"Percentil 95 de diferencias: {np.percentile(corr_diff_flat, 95):.3f
```




```
ANÁLISIS DE DIFERENCIAS EN CORRELACIÓN:
Diferencia máxima en correlación: 0.584
Diferencia promedio: 0.160
Percentil 95 de diferencias: nan
```

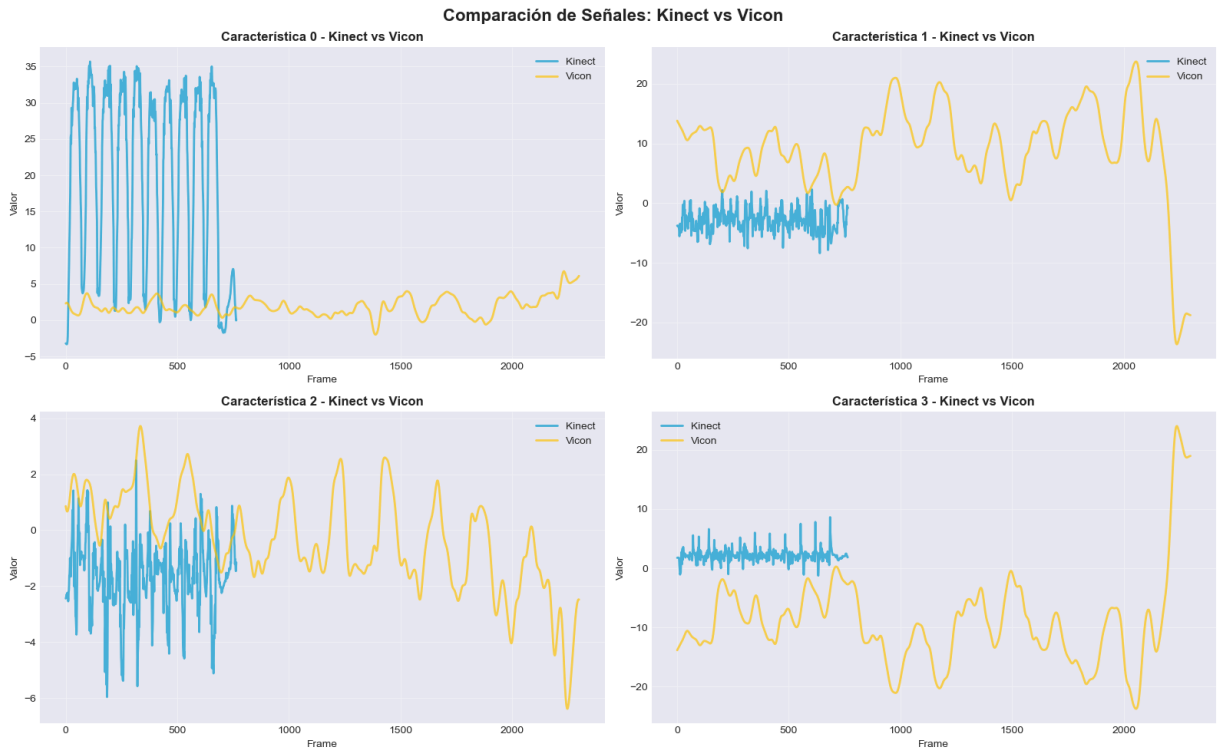
# 10. Comparación entre Sistemas: Kinect vs Vicon

Comparamos las mediciones del mismo movimiento capturado por Kinect y Vicon para evaluar diferencias entre sistemas.

In [54]:
```
# Cargar datos del mismo movimiento de ambos sistemas
kinect_correct = load_category_data("Movements", "Kinect", "Angles", limit=3)
vicon_correct = load_category_data("Movements", "Vicon", "Angles", limit=3)

if kinect_correct and vicon_correct:
    print(f"Cargados {len(kinect_correct)} archivos de Kinect y {len(vicon_correct)

    # Comparar el primer archivo de cada sistema
```

```
    kinect_info, kinect_data = kinect_correct[0]
    vicon_info, vicon_data = vicon_correct[0]

    print(f"Comparación Kinect vs Vicon:")
    print(f"Kinect - Archivo: {kinect_info['filename']}")
    print(f"  Shape: {kinect_data.shape} (frames × características)")
    print(f"  Features: {kinect_data.shape[1]}")
    print(f"\nVicon - Archivo: {vicon_info['filename']}")
    print(f"  Shape: {vicon_data.shape} (frames × características)")
    print(f"  Features: {vicon_data.shape[1]}")

    # Visualizar comparación de las primeras características
    n_features = min(4, kinect_data.shape[1], vicon_data.shape[1])

    fig, axes = plt.subplots(2, 2, figsize=(16, 10))
    axes = axes.flatten()

    for i in range(n_features):
        # Normalizar longitudes para comparación
        kinect_col = kinect_data[i].values
        vicon_col = vicon_data[i].values

        # Graficar ambas señales
        axes[i].plot(kinect_col, label='Kinect', color=PRIMARY_COLOR, alpha=0.7, li
        axes[i].plot(vicon_col, label='Vicon', color=SECONDARY_COLOR, alpha=0.7, li

        axes[i].set_title(f'Característica {i} - Kinect vs Vicon', fontsize=12, fon
        axes[i].set_xlabel('Frame')
        axes[i].set_ylabel('Valor')
        axes[i].legend()
        axes[i].grid(True, alpha=0.3)

    plt.suptitle('Comparación de Señales: Kinect vs Vicon', fontsize=16, fontweight
    plt.tight_layout()
    plt.show()

    print("\nOBSERVACIÓN: Vicon es un sistema óptico de alta precisión, mientras qu
    print("   Las diferencias en el número de features y precisión son esperadas.")
```

```
Cargados 3 archivos de Movements/Kinect/Angles
Cargados 3 archivos de Movements/Vicon/Angles
Cargados 3 archivos de Kinect y 3 de Vicon

Comparación Kinect vs Vicon:
Kinect - Archivo: m01_s01_angles.txt
  Shape: (765, 66) (frames × características)
  Features: 66

Vicon - Archivo: m01_s01_angles.txt
  Shape: (2300, 117) (frames × características)
  Features: 117
```



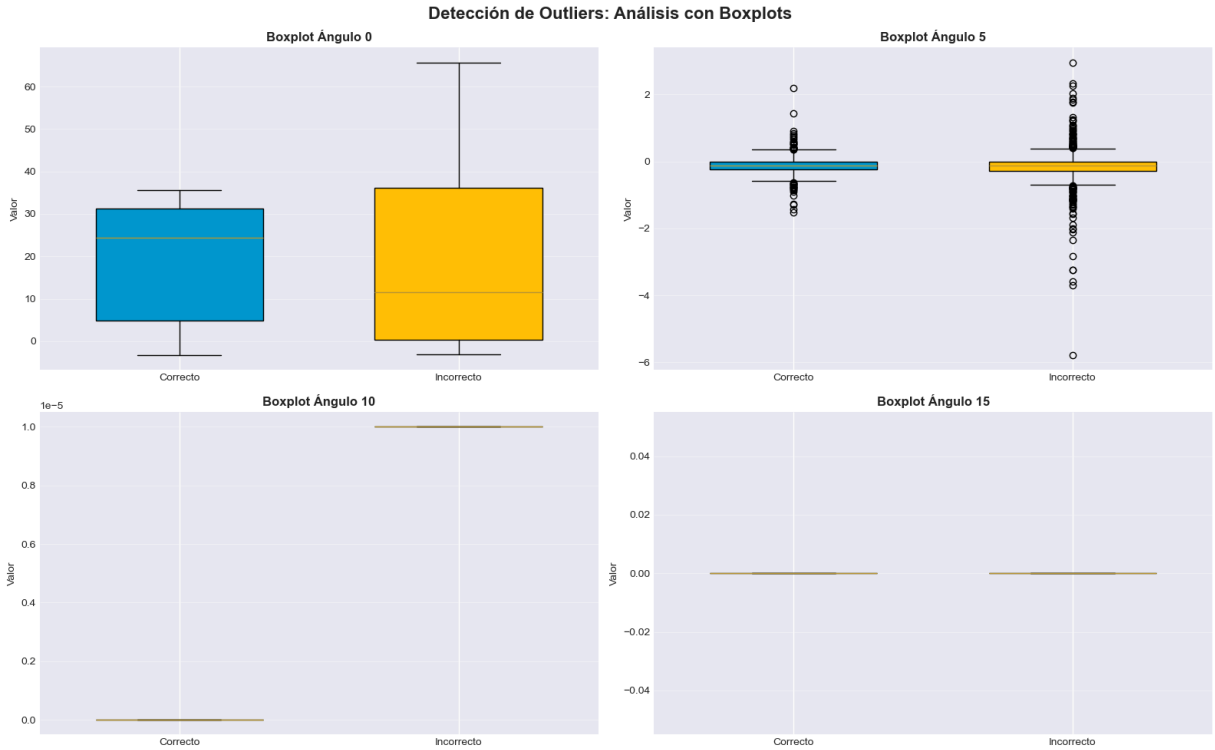

```
OBSERVACIÓN: Vicon es un sistema óptico de alta precisión, mientras que Kinect es un
a cámara de bajo costo.
   Las diferencias en el número de features y precisión son esperadas.
```

# 11. Detección de Outliers y Valores Anómalos

Identificamos valores atípicos que podrían indicar errores de captura o características distintivas de movimientos incorrectos.

In [55]:
```
# Análisis de outliers usando el método IQR
if correct_data and incorrect_data:
    _, correct_sample = correct_data[0]
    _, incorrect_sample = incorrect_data[0]

    # Seleccionar algunas características para análisis
    features_analyze = [0, 5, 10, 15]

    fig, axes = plt.subplots(2, 2, figsize=(16, 10))
    axes = axes.flatten()

    outlier_stats = []

    for i, feature in enumerate(features_analyze):
        if feature < correct_sample.shape[1]:
            # Boxplots lado a lado
            data_to_plot = [correct_sample[feature].values,
                            incorrect_sample[feature].values if feature < incorrect_

            bp = axes[i].boxplot(data_to_plot, labels=['Correcto', 'Incorrecto'],
                                 patch_artist=True, widths=0.6)

            # Colorear
```

```
            bp['boxes'][0].set_facecolor(PRIMARY_COLOR)
            bp['boxes'][1].set_facecolor(SECONDARY_COLOR)

            axes[i].set_title(f'Boxplot Ángulo {feature}', fontsize=12, fontweight=
            axes[i].set_ylabel('Valor')
            axes[i].grid(True, alpha=0.3, axis='y')

            # Calcular outliers (método IQR)
            Q1_correct = correct_sample[feature].quantile(0.25)
            Q3_correct = correct_sample[feature].quantile(0.75)
            IQR_correct = Q3_correct - Q1_correct
            outliers_correct = ((correct_sample[feature] < (Q1_correct - 1.5 * IQR_
                               (correct_sample[feature] > (Q3_correct + 1.5 * IQR_c

            Q1_incorrect = incorrect_sample[feature].quantile(0.25)
            Q3_incorrect = incorrect_sample[feature].quantile(0.75)
            IQR_incorrect = Q3_incorrect - Q1_incorrect
            outliers_incorrect = ((incorrect_sample[feature] < (Q1_incorrect - 1.5
                                 (incorrect_sample[feature] > (Q3_incorrect + 1.5 *

            outlier_stats.append({
                'Ángulo': feature,
                'Outliers_Correcto': outliers_correct,
                'Outliers_Incorrecto': outliers_incorrect,
                'Total_Correcto': len(correct_sample[feature]),
                'Total_Incorrecto': len(incorrect_sample[feature])
            })

    plt.suptitle('Detección de Outliers: Análisis con Boxplots', fontsize=16, fontw
    plt.tight_layout()
    plt.show()

    # Mostrar estadísticas
    outlier_df = pd.DataFrame(outlier_stats)
    outlier_df['%_Correcto'] = (outlier_df['Outliers_Correcto'] / outlier_df['Total
    outlier_df['%_Incorrecto'] = (outlier_df['Outliers_Incorrecto'] / outlier_df['T

    print("\nESTADÍSTICAS DE OUTLIERS:")
    print(outlier_df[['Ángulo', 'Outliers_Correcto', '%_Correcto', 'Outliers_Incorr
```




```
ESTADÍSTICAS DE OUTLIERS:
   Ángulo  Outliers_Correcto  %_Correcto  Outliers_Incorrecto  %_Incorrecto
0       0                 0        0.00                    0          0.00
1       5                42        5.49                  110         10.04
2      10                 0        0.00                    0          0.00
3      15                 0        0.00                    0          0.00
```

# 12. Resumen y Conclusiones del EDA

## Hallazgos Principales:

1. **Estructura del Dataset**
   - 10 movimientos de rehabilitación × 10 sujetos
   - Datos capturados por Kinect y Vicon
   - Incluye ejecuciones correctas e incorrectas
   - Variantes segmentadas y completas
2. **Características de los Movimientos**
   - Los movimientos incorrectos tienden a tener mayor duración
   - Existen diferencias notables en los patrones temporales entre correctos e incorrectos
   - La variabilidad en la ejecución es mayor en movimientos incorrectos
3. **Sistemas de Captura**
   - Kinect y Vicon capturan diferentes números de características
   - Las señales muestran patrones similares pero con diferente precisión
   - Vicon proporciona datos de mayor resolución

4. **Distribuciones y Correlaciones**
   - Las distribuciones de ángulos difieren entre movimientos correctos e incorrectos
   - Existen correlaciones fuertes entre ciertos ángulos articulares
   - Los patrones de correlación varían entre ejecuciones correctas e incorrectas
5. **Outliers**
   - Presencia de valores atípicos en ambos tipos de movimiento
   - Los outliers pueden indicar momentos de transición o errores de captura

## Recomendaciones para Análisis Posteriores:

- 🔹 **Machine Learning**: El dataset es adecuado para clasificación binaria (correcto/incorrecto)
- 🔹 **Feature Engineering**: Considerar derivadas y velocidades angulares
- 🔹 **Segmentación Temporal**: Aprovechar los datos segmentados para análisis detallado de fases
- 🔹 **Análisis Multi-sujeto**: Estudiar variabilidad entre sujetos
- 🔹 **Fusión de Sensores**: Explorar combinación de datos Kinect-Vicon

In [56]:
```
# Resumen ejecutivo de todo el análisis
print("=" * 80)
print(" " * 20 + "RESUMEN EJECUTIVO DEL EDA")
print("=" * 80)

print("\nINFORMACIÓN DEL DATASET:")
print(f"   • Nombre: UI-PRMD (University of Idaho - Physical Rehabilitation Movemen
print(f"   • Movimientos: 10 tipos diferentes")
print(f"   • Sujetos: 10 participantes saludables")
print(f"   • Sistemas: Kinect + Vicon")
print(f"   • Tipos de datos: Ángulos y Posiciones")

print("\n ARCHIVOS ANALIZADOS:")
total_files = 0
for category in ['Movements', 'Incorrect Movements', 'Segmented Movements', 'Incorr
    for system in ['Kinect', 'Vicon']:
        for data_type in ['Angles', 'Positions']:
            path = base_path / category / category / system / data_type
            if path.exists():
                count = len(list(path.glob('*.txt')))
                total_files += count

print(f"   • Total de archivos en el dataset: {total_files}")

print("\n ANÁLISIS COMPLETADOS:")
print("   Exploración de estructura de archivos")
print("   Carga y visualización de datos de muestra")
print("   Comparación de duraciones temporales")
print("   Análisis de series temporales")
print("   Distribuciones estadísticas")
print("   Correlaciones entre características")
```

```
print("   Comparación Kinect vs Vicon")
print("   Detección de outliers")
```

```
================================================================================
                    RESUMEN EJECUTIVO DEL EDA
================================================================================

INFORMACIÓN DEL DATASET:
   • Nombre: UI-PRMD (University of Idaho - Physical Rehabilitation Movement Datase
t)
   • Movimientos: 10 tipos diferentes
   • Sujetos: 10 participantes saludables
   • Sistemas: Kinect + Vicon
   • Tipos de datos: Ángulos y Posiciones

 ARCHIVOS ANALIZADOS:
   • Total de archivos en el dataset: 8756

 ANÁLISIS COMPLETADOS:
   Exploración de estructura de archivos
   Carga y visualización de datos de muestra
   Comparación de duraciones temporales
   Análisis de series temporales
   Distribuciones estadísticas
   Correlaciones entre características
   Comparación Kinect vs Vicon
   Detección de outliers
```